4

Modbus overview

Objectives

When you have completed study of this chapter, you will be able to:

- List the main Modbus structures and frames used
- Identify and correct problems with:
  - No response to protocol messages
  - Exception reports
  - Noise

4.1 General overview

Modbus ® is a transmission protocol (note: a *protocol* only), developed by Gould Modicon (now Schneider Electric) for process control systems. It is, however, regarded as a 'public' protocol and has become the *de facto* standard in multi–vendor integration. In contrast to other buses and protocols, no physical (OSI layer 1) interface has been defined.

MODBUS is a simple, flexible, publicly published protocol, which allows devices to exchange discrete and analog data. End users are aware that specifying MODBUS as the required interface between subsystems is a way to achieve multi–vendor integration with the most purchasing options and at the lowest cost. Small equipment makers are also aware that they must offer MODBUS with EIA–232 and/or EIA–485 to sell their equipment to system integrators for use in larger projects.

System integrators know that MODBUS is a safe interface to commit to, as they can be sure of finding enough equipment on the market to both realize the required designs and handle the inevitable 'change orders,' which come along. However, Modbus suffers from the limitations imposed by EIA–232/485 serial links, including the following:

- Serial lines are relatively slow – 9600 to 115,000 baud means only 0.010 Mbps to 0.115 Mbps. Compare that to today's common 'control network' speeds of 5 to 16 Mbps – or even the new Ethernet speeds of 100 Mbps, and 1Gbps and 10 Gbps!

- While it is easy to link 2 devices by EIA–232 and 20–30 devices by EIA–485, the only solution to link 500 devices with EIA–485 is a complex hierarchy of masters and slaves in a deeply nested tree structure. Such solutions are never simple or easy to maintain.
- Serial links with Modbus are inherently single–master designs. That means, only one device can talk to a group of slave devices – so only that one device (the master) is aware of all the current real–time data.

Designers share this data with multiple operator workstations, control systems, database systems, customized process optimizing workstations and all the other potential users of the data by ending up with complex, fragile hierarchies of master/slave groups shuffling data, up the ladder. Apart from the complexity involved, lower levels of the hierarchy (even expensive DCS systems) waste valuable time shuffling data solely for the benefit of higher levels of the hierarchy.

Even with all these limitations, Modbus has the advantage of wide acceptance among instrument manufacturers and users with many systems in operation. It can therefore be regarded as a *de facto* industrial standard with proven capabilities.

Certain characteristics of the Modbus protocol are fixed, such as frame format, frame sequences, handling of communications errors and exception conditions and the functions performed. Other characteristics are selectable. These are transmission medium, transmission characteristics and transmission mode, viz. RTU or ASCII. The user characteristics are set at each device and cannot be changed when the system is running.

The two transmission modes in which data is exchanged are:

- ASCII – readable; used, for example, for testing. (ASCII format)
- RTU – compact and faster; used for normal operation. (Hexadecimal format)

The RTU mode (sometimes also referred to as Modbus–B for Modbus Binary) is the preferred Modbus mode. The ASCII transmission mode (sometimes referred to as Modbus-A) has a typical message that is about twice the length of the equivalent RTU message.

Modbus also provides an error check for transmission and communication errors. Communication errors are detected by character framing, a parity check, a redundancy check or a sixteen bit cyclic redundancy check (CRC-16). The latter varies depending on whether the RTU or ASCII transmission mode is being used.

Modbus packets can also be sent over local area and wide area networks by encapsulating the Modbus data in a TCP/IP packet.

4.2 Modbus protocol structure

The following illustrates a typical Modbus message frame format.

Address field	Function field	Data field	Error check field
1 byte	1 byte	Variable	2 bytes

Table 4.1
Format of Modbus message frame

The first field in each message frame is the address field, which consists of a single byte of information. In request frames, this byte identifies the controller to which the request is being directed. The resulting response frame begins with the address of the responding device. Each slave can have an address field between 1 and 247; although practical limitations will limit the maximum number of slaves. A typical Modbus installation will have one master and two or three slaves.

The second field in each message is the function field, which also consist of a single byte of information. In a host request, this byte identifies the function that the target PLC is to perform.

If the target PLC is able to perform the requested function, the function field of its response will echo that of the original request. Otherwise, the function field of the request will be echoed with its most–significant bit set to one, thus signaling an exception response. Table 4.2 summarizes the typical functions used.

The third field in a message frame is the data field, which varies in length according to which function is specified in the function field. In a host request, this field contains information the PLC may need to complete the requested function. In a PLC response, this field contains any data requested by that host.

The last two bytes in a message frame comprise the error–check field. The numeric value of this field is calculated by performing a Cyclic Redundancy Check (CRC–16) on the message frame. This error checking assures that devices do not react to messages that may have been damaged during transmission.

Table 9.2 lists the address range and offsets for these four data types, as well as the function codes that apply to each. The diagram above also gives an easy reference to the Modbus data types.

Data type	Absolute addresses	Relative addresses	Function codes	Description
Coils	00001 to 09999	0 to 9998	01	Read coil status
Coils	00001 to 09999	0 to 9998	05	Force single coil
Coils	00001 to 09999	0 to 9998	15	Force multiple coils
Discrete inputs	10001 to 19999	0 to 9998	02	Read input status
Input registers	30001 to 39999	0 to 9998	04	Read input registers
Holding registers	40001 to 49999	0 to 9998	03	Read holding register
Holding registers	40001 to 49999	0 to 9998	06	Preset single register
Holding registers	40001 to 49999	0 to 9998	16	Preset multiple registers
–	–	–	07	Read exception status
–	–	–	08	Loopback diagnostic test

Table 4.2
Modicon addresses and function codes

4.3 Function codes

Each request frame contains a function code that defines the action expected for the target controller. The meaning of the request data fields is dependent on the function code specified.

The following paragraphs define and illustrate most of the popular function codes supported. In these examples, the contents of the message–frame fields are shown as hexadecimal bytes.

4.3.1 Read coil or digital output status (Function code 01)

This function allows the host to obtain the ON/OFF status of one or more logic coils in the target device.

The data field of the request consists of the relative address of the first coil followed by the number of coils to be read. The data field of the response frame consists of a count of the coil bytes followed by that many bytes of coil data.

The coil data bytes are packed with one bit for the status of each consecutive coil (1=ON, 0=OFF). The least significant bit of the first coil data byte conveys the status of the first coil read. If the number of coils read is not an even multiple of eight, the last data byte will be padded with zeros on the high end. Note that if multiple data bytes are requested, the low order bit of the first data byte in the response of the slave contains the first addressed coil.

In the following example, the host requests the status of coils 000A (decimal 00011) and 000B (decimal 00012). The target device's response indicates both coils are ON.

Request Message

Address	Function Code	Initial Coil Offset		Number of Points		CRC
		Hi	Lo	Hi	Lo	
01	01	00	0A	00	02	9D C9

Response Frame

Address	Function Code	Byte Count	Coil Data	CRC
01	01	01	03	11 89

Figure 4.1
Example of read coil status

4.3.2 Read digital input status (Function code 02)

This function enables the host to read one or more discrete inputs in the target device.

The data field of the request frame consists of the relative address of the first discrete input followed by the number of discrete inputs to be read. The data field of the response frame consists of a count of the discrete input data bytes followed by that many bytes of discrete input data.

The discrete–input data bytes are packed with one bit for the status of each consecutive discrete input (1=ON, 0=OFF). The least significant bit of the first discrete input data byte conveys the status of the first input read. If the number of discrete inputs read is not an even multiple of eight, the last data byte will be padded with zeros on the high end. The low order bit of the first byte of the response from the slave contains the first addressed digital input.

In the following example, the host requests the status of discrete inputs with offsets 0000 and 0001 hex i.e. decimal 10001 and 10002. The target device's response indicates that discrete input 10001 is OFF and 10002 are ON.

Request Message

Address	Function Code	Initial Coil Offset		Number of Points		CRC
		Hi	Lo	Hi	Lo	
01	02	00	00	00	02	F9 CB

Response Frame

Address	Function Code	Byte Count	Input Data	CRC
01	02	01	02	20 49

Figure 4.2
Example of read input status

4.3.3 Read holding registers (Function code 03)

This function allows the host to obtain the contents of one or more holding registers in the target device.

The data field of the request frame consists of the relative address of the first holding register followed by the number of registers to be read. The data field of the response time consists of a count of the register data bytes followed by that many bytes of holding register data.

The contents of each requested register (16 bits) are returned in two consecutive data bytes (most significant byte first).

In the following example, the host requests the contents of holding register hexadecimal offset 0002 or decimal 40003. The controller's response indicates that the numerical value of the register's contents is hexadecimal 07FF or decimal 2047. The first byte of the response register data is the high order byte of the first addressed register.

Request Message

Address	Function Code	Starting Register		Register Count		CRC	
		Hi	Lo	Hi	Lo		
01	03	00	02	00	01	25	CA

Response Frame

Address	Function Code	Byte Count	Register Data		CRC	
			Hi	Lo		
01	03	02	07	FF	FA	34

Figure 4.3
Example of Reading Holding Register

4.3.4 Reading input registers (Function code 04)

This function allows the host to obtain the contents of one or more input registers in the target device.

The data field of the request frame consists of the relative address of the first input register followed by the number of registers to be read. The data field of the response frame consists of a count of the register–data bytes followed by that many bytes of input–register data.

The contents of each requested register are returned in two consecutive register–databytes (most–significant byte first). The range for register variables is 0 to 4095.

In the following example, the host requests the contents of input register hexadecimal offset 000 or decimal 30001. The PLC's response indicates that the numerical value of that register's contents is 03FFH, which would correspond to a data value of 25 percent (if the scaling of 0 to 100 percent is adopted) and a 12 bit D to A converter with a maximum reading of 0FFFH is used.

Request Message

Address	Function Code	Starting Register		Register Count		CRC	
		Hi	Lo	Hi	Lo		
01	04	00	00	00	01	31	CA

Response Frame

Address	Function Code	Byte Count	Register Data		CRC	
			Hi	Lo		
01	04	02	03	FF	F9	80

Figure 4.4
Example of Reading Input Register

4.3.5 Force single coil (Function code 05)

This function allows the host to alter the ON/OFF status of a single logic coil in the target device.

The data field of the request frame consists of the relative address of the coil followed by the desired status for that coil. A hexadecimal status value of FF00 will activate the coil, while a status value of 0000H will deactivate it. Any other status value is illegal.

If the controller is able to force the specified coil to the requested state, the response frame will be identical to the request. Otherwise, an exception response will be returned.

If the address 00 is used to indicate broadcast mode, all attached slaves will modify the specified coil address to the state required.

The following example illustrates a successful attempt to force coil 11 (decimal) OFF.

Request Message

Address	Function Code	Coil Offset		New Coil Status		CRC	
		Hi	Lo	Hi	Lo		
01	05	00	0A	00	00	ED	C8

Response Frame

Address	Function Code	Coil Offset		New Coil Status		CRC	
		Hi	Lo	Hi	Lo		
01	05	00	0A	00	00	ED	C8

Figure 4.5
Example of forcing a single coil

4.3.6 Preset single register (Function code O6)

This function enables the host to alter the contents of a single holding register in the target device.

The data field of the request frame consists of the relative address of the holding register followed by the new value to be written to that register (most–significant byte first).

If the controller is able to write the requested new value to the specified register, the response frame will be identical to the request. Otherwise, an exception response will be returned.

The following example illustrates a successful attempt to change the contents of holding register 40003 to 3072 (0C00 Hex).

When slave address is set to 00 (broadcast mode), all slaves will load the specified register with the value specified.

Request Message

Address	Function Code	Register Offset		Register Value		CRC	
		Hi	Lo	Hi	Lo		
01	06	00	02	0C	00	2D	0A

Response Frame

Address	Function Code	Register Offset		Register Value		CRC	
		Hi	Lo	Hi	Lo		
01	06	00	02	0C	00	2D	0A

Figure 4.6
Example of Presetting a Single Register

4.3.7 Read exception status (Function code 07)

This is a short message requesting the status of eight digital points within the slave device.

This will provide the status of eight predefined digital points in the slave. For example, this could be items such as the status of the battery, whether memory protect has been enables or the status of the remote input/output racks connected to the system.

Request Message

Address	Function Code	CRC	
11	07	..	..

Response Frame

Address	Function Code	Coil Station	CRC	
11	07	02	..	..

Figure 4.7
Read exception status query message

4.3.8 Loopback test (Function code 08)

The objective of this function code is to test the operation of the communications system without affecting the memory tables of the slave device. It is also possible to implement additional diagnostic features in a slave device (should this be considered necessary) such as number of CRC errors, number of exception reports etc.

The most common implementation will only be considered in this section; namely, a simple return of the query messages.

Request Frame

Address	Function Code	Data Diagnostic Code		Data		CRC
		Hi	Lo	Hi	Lo	
11	08	00	00	A5	37	.. ..

Response Frame

Address	Function Code	Data Diagnostic Code		Data		CRC
		Hi	Lo	Hi	Lo	
11	08	00	00	A5	37	.. ..

Figure 4.8
Loopback test message

4.3.9 Force multiple coils or digital outputs (Function code 0F)

This forces a contiguous (or adjacent) group of coils to an ON or OFF state. The following example sets 10 coils starting at address 01 Hex (at slave address 01) to the ON state. If slave address 00 is used in the request frame, broadcast mode will be implemented resulting in all slaves changing their coils at the defined addresses.

Request Frame

Address	Function Code	Address	Byte Count	Data Coil Status	CRC
		Hi Lo		Hi Lo	
01	0F	00 01	0F	FF 03	.. ..

Response Frame

Address	Function Code	Address	Number of Coils	CRC
		Hi Lo	Hi Lo	
01	0F	00 01	00 0A	.. ..

Figure 4.9
Example of forcing multiple coils

4.3.10 Force multiple registers (Function code 10)

This is similar to the preset a single register and the forcing of multiple coils. In the example below, a slave address 01 has 2 registers changed commencing at address 10.

Request Frame

Address	Function Code	Address	Quantity	Byte Count	First Register	Second Register	CRC
		Hi Lo	Hi Lo		Hi Lo	Hi Lo	
01	10	00 0A	00 02	04	00 0A	01 02	.. ..

Response Frame

Address	Function Code	Address	Quality	CRC
		Hi Lo	Hi Lo	
01	10	00 0A	00 02	.. ..

Figure 4.10
Example of Presetting Multiple Registers

Table 4.3 lists the most important exception codes that may be returned.

Code	Name	Description
01	Illegal function	Requested function is not supported.
02	Illegal data address	Requested data address is not supported
03	Illegal data value	Specified data value is not supported
04	Failure in associated device	Slave PLC has failed to respond to a message
05	Acknowledge	Slave PLC is processing the command
06	Busy, rejected message	Slave PLC is busy

Table 4.3
Abbreviated list of exception codes returned

An example of an illegal request and the corresponding exception response is shown below. The request in this example is to READ COIL STATUS of points 514 to 521 (eight coils beginning with an offset 0201H). These points are not supported in this PLC, so an exception report is generated indicating code 02 (illegal address).

Request Message

Address	Function Code	Starting Point	Number of Points	CRC
01	01	02 01	00 08	6D B4

Exception Response Message

Address	Function Code	Exception Code	CRC
01	81	02	C1 91

Figure 4.11
Example of an illegal request

4.4 Common problems and faults

No matter what extremes of care you may have taken, there is hardly ever an installation that boasts of trouble-free setup and configuration. Some of the commonly faced

problems and faults that one comes across in the industry are listed below. This list is broken into two distinct groups:

4.4.1 Hardware related

This includes mis-wired communication cabling and faulty communication interfaces.

4.4.2 Software related

Software related issues arise when the master application tries to access non-existent nodes or use invalid function codes, address non-existent memory locations in the slaves, or specify illegal data format types, which obviously the slave devices do not understand. These issues will be dealt with in detail under the Modbus plus protocol, as these issues are common to both the traditional Modbus protocol and the Modbus plus protocol. They are summarized under software related problems. These issues are also applicable to the latest Modbus/TCP protocol.

4.5 Description of tools used

In order to troubleshoot the problems listed above, one would require the use of a few tools, hardware and software, to try and decipher the errors. The most important tool of all would always be the instruction manuals of the various components involved.

The hardware tools that may be required include RS-232 break-out boxes, RS-232 to RS-485 converters, continuity testers, voltmeters, screw drivers, pliers, crimping tools, spare cabling and other similar tools & tackle. These would generally be used to ensure that the cabling and terminations are proper and have been installed as per the recommended procedures detailed in the instruction manuals.

On the software front, one would need some form of protocol analyzer that is in a position to eavesdrop on the communications link between the master and slave modules. This could be either a dedicated hardware protocol analyzer, that is very expensive, or a software based protocol analyzer that could reside on a computer.

Obviously, this second option is more economical and also requires the relevant hardware component support in order to connect to the network.

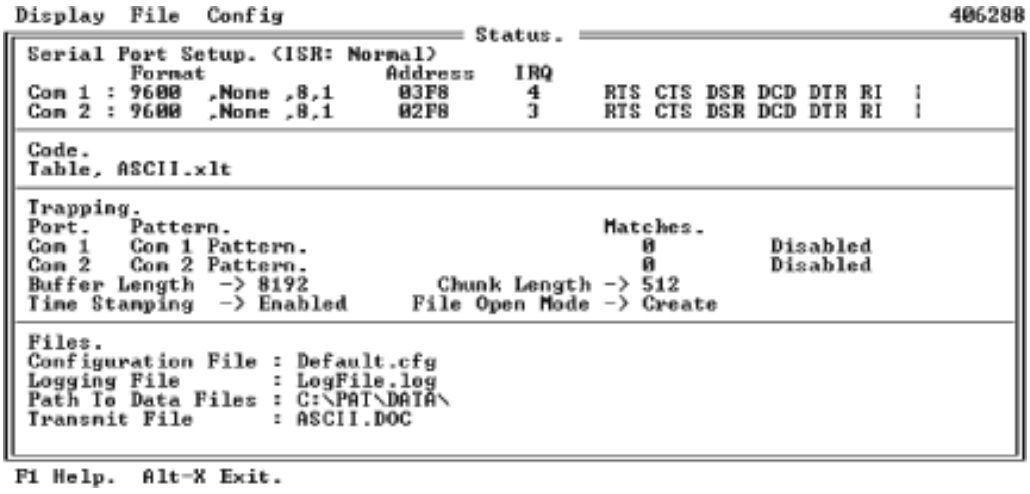


Figure 4.12
Screen shot of the protocol analysis tool

4.6 Detailed troubleshooting

4.6.1 Mis-wired communication cabling

RS-232 wiring for 3-wire point-to-point mode

There are various wiring scenarios such as point-to-point, multi-drop two-wire, multi-drop 4-wire, etc. In a point-to-point configuration, the physical standard used is usually RS-232 on a 25 Pin D connector, which means a minimum of three wires are used. They are: on the DTE (master) side: transmit (TxD – pin 2), receive (RxD – pin 3) and signal ground (common – pin 7); and on the DCE (slave) side: receive (RxD – pin 2), transmit (TxD – pin 3) and signal ground (common – pin 7).

The other pins were primarily used for handshaking between the two devices for data flow control. Nowadays, these pins are rarely used as the flow control is done via software handshaking protocols.

With the advent of VLSI technology, the footprint of the various devices have a tendency to shrink; thereby even the physical connections of communications have been now been standardized on 9-pin D connectors. The pin assignment that has been adopted is the IBM standard in which the pin configurations are: on the DTE (master) side: transmit (TxD – pin 3), receive (RxD – pin 2) and signal ground (common – pin 5); and on the DCE (Slave) side: receive (RxD – pin 3), transmit (TxD – pin 2) and signal ground (common – pin 5).

It follows that the cabling between the two devices must be straight-through pin-for-pin. In this manner, the transmit pin of the master is directly connected to the receive pin of the slave and vice-versa. These cables are standard off-the-shelf products available in standard predetermined lengths. They can also be fabricated to custom lengths, with ease.

Master devices are usually the present day IBM compatible computers, with a Modbus application enabled on it, and therefore have the standard IBM RS-232 port provided. The slave devices usually have a user selectable option to have either an RS-232 or RS 485 port for communication. Unfortunately some manufacturers, in order to force the customers to return to them time-and-again, employ a strategy of modifying these standards to their own advantage.

Illustrated below are a couple of these combinations:

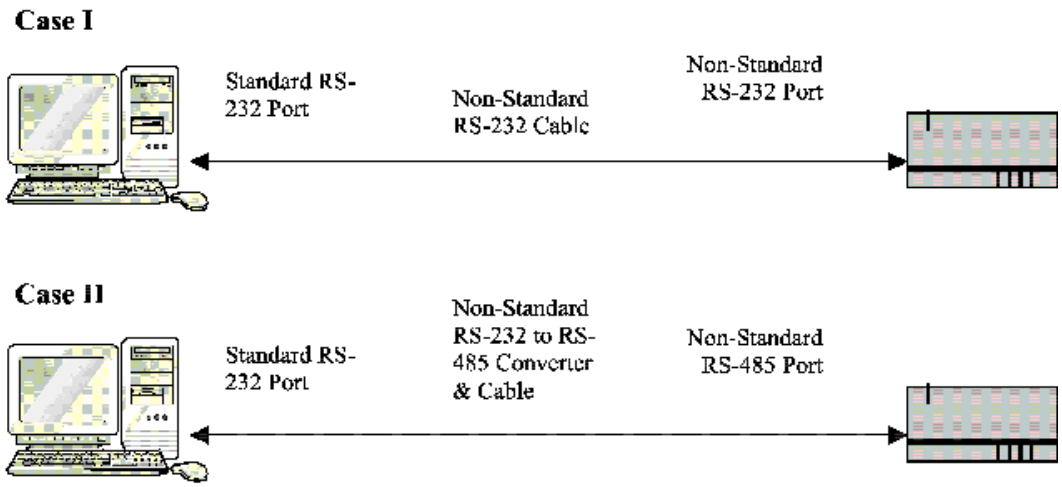


Figure 4.13
Typical customizations in RS-232 cabling

As can be seen in case I, the manufacturer has modified the standard pin-out of the RS-232 connection or has totally replaced the standard IBM RS-232 9-pin DIN connector with his own standard. In situations like these, it is then imperative to possess the manufacturer's manuals of the device being used.

With the aid of the continuity checker, and the use of the RS-232 breakout box, you can determine the non-standard pin-out of the cable, and fabricate a spare cable to the required lengths without having to revert to the supplier.

As in case II, the manufacturer has embedded an RS-232 to RS-485 converter into the cable itself and this would mimic a standard off the shelf RS-232 serial cable. This obviously would not be clearly evident if you were not aware of the modification. With the aid of the voltage tester, you can determine the operating voltages and therefore be in a position to decipher the type of standard being employed.

In other implementations, where multi-dropped RS-485 is used, the installations usually cater for both configurations of two-wire and four-wire communications. In the case of single master configurations, either of the two wiring modes could be used. In the case of multi master configurations, only the two-wire communication can be used.

RS-485/RS-422 wiring for 4-wire, repeat or multi-drop modes

The four-wire configuration is, in actual fact, the RS-422 standard where there is one line driver connected to multiple receivers, or multiple line drivers connected to one line receiver. When using 4-wire RS-422 communications, messages are transmitted on one pair of shielded, twisted wires and received on another (Belden part number 9729, 9829, or equivalent). Both multi-drop and repeat configurations are possible when RS-422 is used.

This is the classic case where the one line driver is the one that belongs to the master and the remaining receivers belong to the multiple slaves. The slaves receive their commands via this link and respond via their line drivers that are all connected to each other and finally connected to the master's receiver.

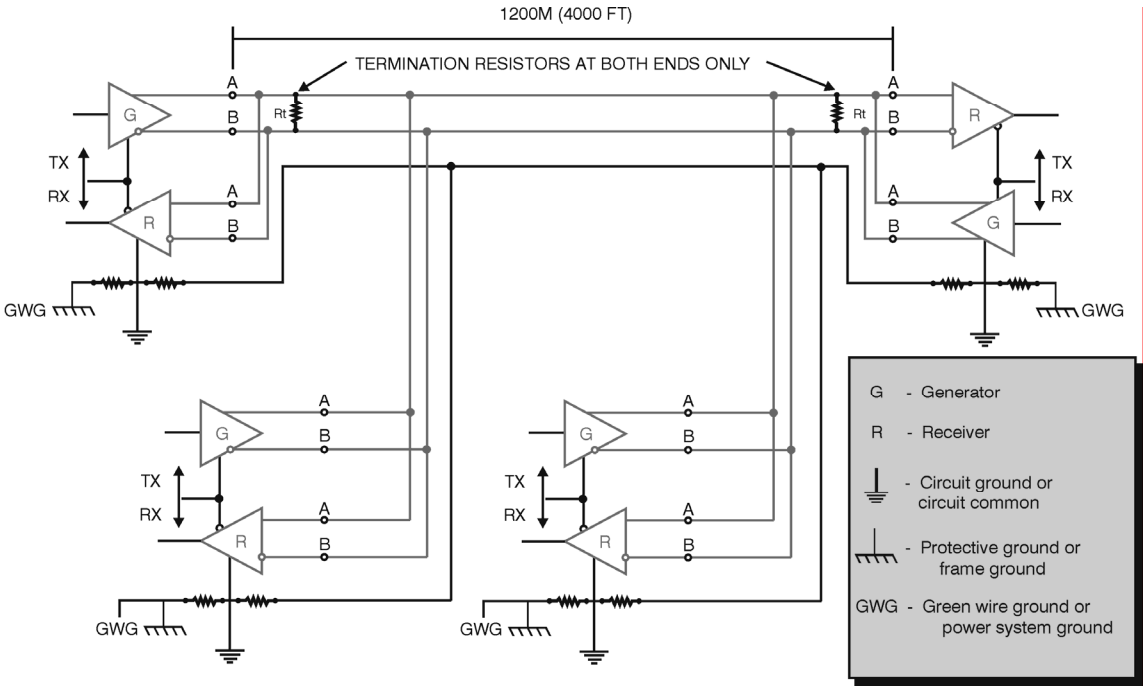


Figure 4.14
Typical two-wire RS-422/485 cabling

RS-485 wiring for 2-wire, multidrop mode only

The two-wire configuration is the actual RS-485 standard where both the line drivers and the receivers are connected to the same pair of communication cables. When using 2-wire RS-485 communications, messages are transmitted and received on the same pair of shielded, twisted wires. Care must be taken to turn the host driver IC on when sending messages and off when receiving messages. This can be accomplished using software or hardware. Only multi-drop wiring configuration is possible when RS-485 is used.

Typically, the maximum number of physical RS-485 nodes on a network is 32. If more physical nodes are placed on the network an RS-485 repeater must be used.

This system works inherently similar to the four-wire system except that the master transmits requests to the various slaves and the slaves respond to the master over the same pair of wires.

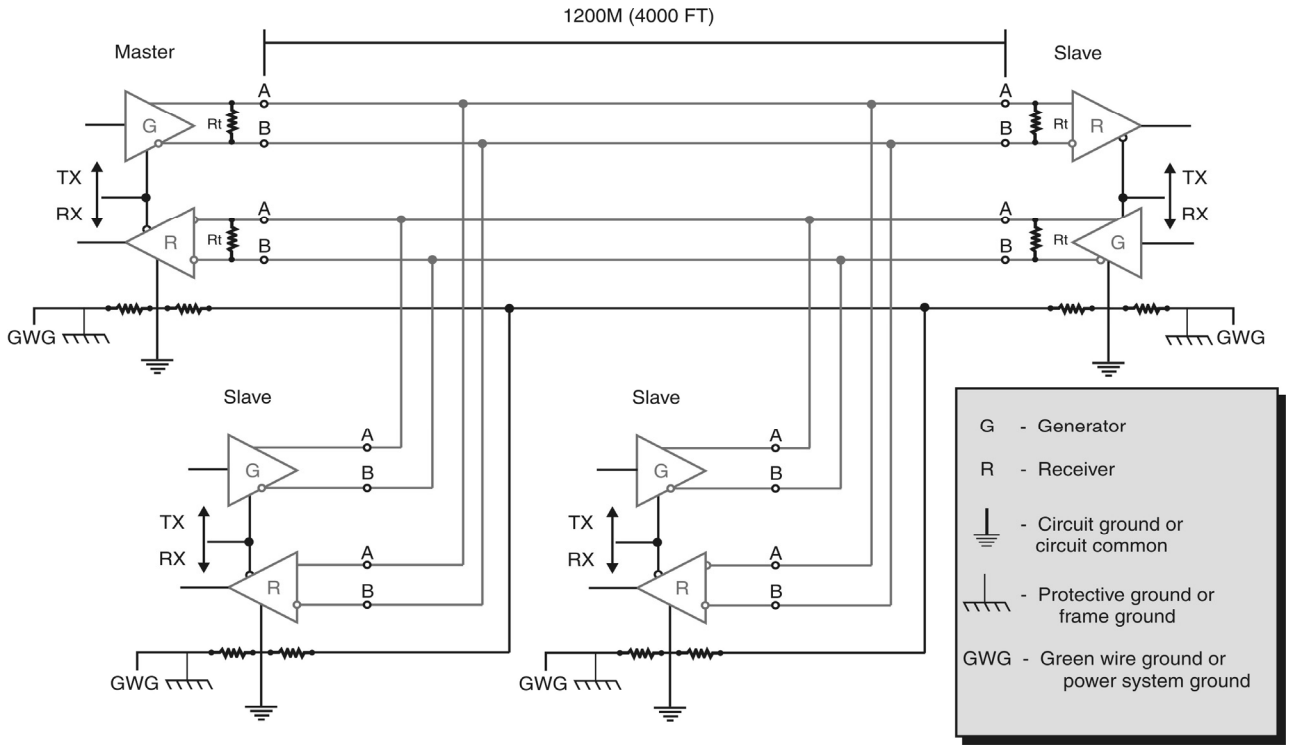


Figure 4.15
Typical four-wire RS-485 multi-drop cabling

Note: The shield grounds should be connected to system ground at one point only. This will eliminate the possibility of ground loops causing erroneous operation.

Grounding

The logic ground of all controllers and any other communication device on the same network must reference the same ground. If one power supply is used for all controllers, then they will be referenced to the same ground. When multiple supplies are used, there are a couple of options to consider:

- Connect the supply output ground to a solid earth ground on all power supplies. If the grounding system is good, all controllers will be referenced to the same ground. (Note that the typical PC follows this practice).

In some older buildings, the earth ground could vary by several volts within the building. In this case, option 2 should be considered

- Use the GND position on each controller to tie all controllers to the same potential. Note that each power supply output must be isolated from the input lines, otherwise irreparable damage may result if two power supplies are powered from different phases of an electrical distribution system. It is not recommended that the shield of a cable be used for this purpose, use a separate conductor
- Where it is not possible to get all controllers referenced to the same ground, use an isolated RS-485 repeater between controllers that are located at different ground potentials. Isolated repeaters are also an excellent way to clean up signals and extend distances in noisy environments

4.6.2 Faulty communication interfaces

RS-232 driver failed

The RS-232 driver may be tested with a good high impedance voltmeter. Place the meter in the DC voltage range. Place the RED probe on the transmit pin (TxD) and the BLACK probe on the signal ground (common). While the node is not transmitting (TX light off), there should be a voltage between –5 VDC and –25 VDC across these pins. This is referred to as the idle state of the RS-232 driver. When the node is transmitting, the voltage will oscillate through between ±5VDC and ±25 VDC. It may be difficult to see the deflection at the higher baud rates and an oscilloscope is suggested for advanced troubleshooting. If the voltage is fixed at anywhere between –25 VDC and +25 VDC and doesn't oscillate as the TX light blinks, then the transmitter is probably damaged and must be returned to the factory for repair or replacement.

RS-232 receiver failed

This requires the use of an RS-232 loop back plug or an RS-232 breakout box with a jumper installed between the transmit and receive pins. After connecting this to the communication port of the node to be tested, the node is made to transmit data, and when the TxD light of the node flashes, the RxD light must also flash, then it can be said that the RS-232 receiver of that node is good. If the RxD light does not flash at the same time as the TxD, then the RS-232 receiver may be bad.

Alternatively, two nodes may be connected to each other with a tested and working communication cable and both their RS-232 drivers working. If one of the nodes is transmitting to the other, the second node indicates a good reception by flashing its RxD light as the first node transmits, then the RS-232 receiver on the second node is good. The same is true when the second node transmits and the first flashes its RxD light.

RS-485 driver failed

The RS-485 driver too may be tested with a good high impedance voltmeter. Place the meter in the DC voltage range. Place the RED probe on the non-inverted transmit terminal (TX+) and the BLACK probe on the inverted transmit terminal (TX–). While the node is not transmitting (TX light off), there should be approximately minus 4 Vdc across these wires. When the node is transmitting, the voltage will oscillate through ±4Vdc. It may be difficult to see the deflection at the higher baud rates and an oscilloscope is suggested for advanced troubleshooting. The minimum voltage obtained during a full 1200 baud to 19200 baud sweep will be around –1.6 VDC. If the voltage is fixed at

around +4, 0, or –4 volts, and doesn't oscillate as the TX light blinks, then the transmitter is probably damaged and must be returned to the factory for repair or replacement.

RS-485 receiver failed

This requires two nodes connected to each other with a tested and working communication cable and both their RS-485 drivers working. If one of the nodes is transmitting to the other, the second node indicates a good reception by flashing its RX light as the first node transmits, then the RS-485 receiver on the second node is good. If the RX light flashes when the first node polls at other baud rates then most likely, the polarity is reversed between the OUT of the first node and the second node. Reverse the TX+ and TX– wires at the first node.

4.6.3 Software related problems

See the suggestions under Modbus Plus. But the important issues to consider are:

No response to message from master to slave

This could mean that either the slave does not exist or there are CRC errors in the transmitted message due to noise (or incorrectly formatted message).

Exception responses

See the list of potential problems reported by the exception responses. This could vary from slave address problems to I/O addresses being illegal.

4.7 Conclusion

We have seen that the master computer sends commands to the various slave units to determine the status of its various process inputs or to change the status of its outputs using the Modbus protocol. The commands are transmitted over a single pair of twisted wires (RS-485) or two pair of twisted wires (RS-422) at speeds of 9600, 19200, 38400, or 57600 baud. The addressed slave decodes the commands and returns the appropriate response. If the master computer is an IBM PC or compatible, inexpensive interface driver software is available. This software dramatically simplifies sending and receiving these messages. If you prefer, you can use one of the off-the-shelf graphics-based data acquisition and control software packages. Many of these packages offer a Modbus compatible driver.

If you were to follow the recommended installation and startup procedure and took extra precaution while setting up the following items, then you would achieve trouble free startup:

- Setting the base address
- Setting the protocol and baud rate
- Serial communications wiring
- Communication wiring termination